



CLASIFICACIÓN DE ENFERMEDADES DE PIEL Y CÁNCER CUTÁNEO CON IMÁGENES, APLICANDO DEEP LEARNING

Cátedra: Deep Learning para la Inteligencia Artificial



Apellidos	Nombres	Número de carné
Arévalo Alvarado	Raúl Anibal	AA252630
Cartagena Candelario	Emerson Francisco	CC100308
Olmedo Lopez	Edwin Josue	OL150100
Rivas Araniva	Diego Alfonso	RA162519

Universidad Don Bosco

El Salvador, sábado 18 de abril del 2026

Docente: MSc Joel Orellana

Contenido

1. Introducción.....	2
2. Descripción del problema	2
3. Descripción de Dataset.....	3
Variables del DataSet	4
4. Metodología y Arquitecturas Utilizadas	4
4.1 Fase 1: Modelo Base.....	5
4.2 Etapa 2: Exploración de Arquitecturas Profundas.....	5
4.3 Etapa 3: Transfer Learning con EfficientNetB0	6
4.4 Etapa 4: Componente Generativo	6
5.Experimentos realizados y comparación entre modelos	7
5.1 Configuración experimental	7
5.2 Resultados reales obtenidos	10
5.3 Análisis comparativo de resultados	11
6. Bibliografía	13

1. Introducción

El rápido avance de la inteligencia artificial, y en particular del Deep Learning, ha permitido el desarrollo de soluciones innovadoras en el ámbito de la salud, especialmente en el diagnóstico asistido por imágenes. Una de las áreas donde estas tecnologías han demostrado un gran potencial es en la detección temprana de enfermedades dermatológicas, incluyendo distintos tipos de cáncer de piel.

El cáncer de piel es una de las enfermedades más comunes a nivel mundial, y su detección temprana resulta crucial para aumentar las probabilidades de tratamiento exitoso. Sin embargo, el diagnóstico tradicional depende en gran medida de la experiencia clínica del especialista, lo que puede generar variabilidad en los resultados y limitar el acceso a diagnósticos precisos en regiones con pocos recursos médicos.

En este contexto, el presente proyecto tiene como objetivo desarrollar un modelo basado en técnicas de Deep Learning capaz de clasificar imágenes de enfermedades de la piel, utilizando el dataset “Skin Diseases - Cancer Comprehensive Dataset” disponible en Kaggle. A través del entrenamiento de redes neuronales convolucionales (CNN), se busca construir un sistema automatizado que apoye el diagnóstico médico, mejorando la precisión y reduciendo los tiempos de análisis.

2. Descripción del problema

El diagnóstico de enfermedades dermatológicas, particularmente aquellas relacionadas con el cáncer de piel, presenta diversos desafíos. En primer lugar, muchas de estas patologías comparten características visuales similares, lo que dificulta su identificación precisa incluso para profesionales experimentados. Además, el proceso de diagnóstico suele requerir herramientas especializadas, como biopsias o análisis dermatoscópicos, que no siempre están disponibles en todos los contextos.

Otro problema relevante es la creciente demanda de servicios de salud frente a la limitada disponibilidad de especialistas en dermatología. Esto provoca retrasos en la atención médica, lo que puede derivar en diagnósticos tardíos y, en consecuencia, en un empeoramiento del pronóstico del paciente.

Desde el punto de vista tecnológico, aunque existen grandes volúmenes de datos médicos disponibles, estos no siempre son aprovechados de manera eficiente. La falta de sistemas automatizados capaces de analizar imágenes médicas de forma precisa y rápida representa una oportunidad para la aplicación de modelos de Deep Learning.

Por lo tanto, la problemática central que aborda este proyecto consiste en la necesidad de desarrollar un sistema inteligente que permita la clasificación automática de enfermedades de la piel a partir de imágenes, con el fin de apoyar el diagnóstico clínico, reducir errores humanos y facilitar el acceso a herramientas de detección temprana, especialmente en entornos con recursos limitados.

3. Descripción de Dataset

El dataset utilizado corresponde a un conjunto de datos disponible en Kaggle enfocado en enfermedades de la piel y cáncer cutáneo. Este dataset tiene como objetivo principal servir como base para el desarrollo de modelos de aprendizaje automático, especialmente en tareas de clasificación de imágenes dentro del ámbito de la dermatología.

Este conjunto de datos está compuesto principalmente por imágenes dermatológicas etiquetadas según el tipo de enfermedad presente. Estas imágenes representan diferentes condiciones de la piel, incluyendo tanto afecciones benignas como enfermedades más complejas, entre ellas distintos tipos de cáncer cutáneo. Esto lo convierte en un problema de clasificación multiclase, donde el objetivo es que el modelo aprenda a identificar correctamente la categoría de la enfermedad a partir de características visuales.

Otra característica importante es que los datos presentan una estructura organizada por clases (carpetas por tipo de enfermedad), lo que simplifica su procesamiento y uso en frameworks como TensorFlow o PyTorch. Asimismo, el dataset puede presentar cierto

desbalance entre clases, lo cual representa un desafío adicional y permite aplicar técnicas como data augmentation o generación de datos sintéticos en etapas posteriores del proyecto.

En el contexto del sistema a desarrollar, este dataset permite construir un modelo capaz de asistir en la clasificación automática de enfermedades de la piel, contribuyendo potencialmente a sistemas de apoyo al diagnóstico médico.

Variables del DataSet

El dataset utilizado no presenta una estructura tabular tradicional (como un archivo .csv con columnas y registros). En su lugar, está compuesto por un conjunto de imágenes organizadas en una estructura jerárquica de carpetas. Cada carpeta corresponde a una clase o categoría específica (etiqueta), representando un tipo de enfermedad de la piel. A su vez, cada archivo dentro de estas carpetas corresponde a una imagen individual, la cual constituye una instancia del dataset.

Parámetro	Descripción	Tipo
image	Imagen dermatológica que representa una condición de la piel	Imagen (RGB)
Etiqueta	Categoría de la enfermedad, determinada por la carpeta donde se almacena	Categórica
Clase	Nombre específico de la enfermedad	Cadena
Ruta	Ubicación de la imagen dentro de la estructura de carpetas del dataset	Cadena

4. Metodología y Arquitecturas Utilizadas

El desarrollo del sistema de clasificación de enfermedades dermatológicas se llevó a cabo bajo una metodología incremental, dividida en cuatro etapas principales. Esto permitió evaluar el impacto y la viabilidad de distintas arquitecturas neuronales

4.1 Fase 1: Modelo Base.

Multilayer Perceptron Tabular: Como punto de partida se implementó un Perceptrón Multicapa (MLP) que procesa directamente los píxeles de las imágenes dermatológicas, estableciendo una línea base de desempeño para comparación con etapas posteriores.

Preprocesamiento: Las imágenes se redimensionaron a 192×192 píxeles. Dentro del modelo se aplica reescalado de píxeles al rango [0,1] y augmentación moderada (flip horizontal, rotación, zoom y contraste aleatorio). Un AveragePooling con pool_size=6 reduce la dimensionalidad antes del bloque denso, evitando un número excesivo de parámetros.

Arquitectura: Tres capas densas de 768, 384 y 192 neuronas con activación ReLU, cada una seguida de BatchNormalization y Dropout (0.45, 0.35 y 0.25 respectivamente).

Capa de salida: Densa con activación Softmax en float32 para estabilidad numérica.

Optimización: Optimizador AdamW con weight decay de $2e-4$ y clipnorm=1.0, entrenado con hasta 35 épocas, EarlyStopping (patience=8) y ReduceLROnPlateau. Se utilizó mixed precision (float16) para reducir el uso de memoria en GPU. El desbalance de clases se corrigió mediante class weights suavizados con raíz cuadrada.

4.2 Etapa 2: Exploración de Arquitecturas Profundas

En esta etapa se implementó una Red Neuronal Convolutiva (CNN) profunda entrenada desde cero, sin uso de pesos preentrenados, para aislar el efecto de la arquitectura convolutiva frente al baseline de la etapa anterior.

Arquitectura: Cuatro bloques convolutivos progresivos con 32, 64, 128 y 256 filtros respectivamente. Cada bloque aplica dos capas Conv2D (kernel 3×3, sin bias), BatchNormalization, activación ReLU, MaxPooling2D y Dropout creciente (0.10 a 0.25). Una capa Conv2D adicional de 320 filtros (denominada last_conv_f2) precede al GlobalAveragePooling2D. La cabeza de clasificación incluye una capa densa de 256 neuronas con BatchNormalization y Dropout de 0.45.

El input se aumenta con flip horizontal, rotación, zoom y contraste aleatorio. El modelo opera sobre imágenes de 224×224 píxeles.

Análisis opcional, Grad-CAM: Se implementaron mapas de activación mediante Gradient-weighted Class Activation Mapping sobre la capa `last_conv_f2`, permitiendo visualizar las regiones de la imagen que el modelo priorizó para cada predicción.

Comparación obligatoria: Se realizó una comparación cuantitativa directa entre el MLP base (Etapa 1) y esta CNN en todas las métricas principales.

4.3 Etapa 3: Transfer Learning con EfficientNetB0

Por la complejidad del dominio visual clínico y el desbalance de clases inherente al dataset, se adoptó una estrategia de transferencia de conocimiento (Transfer Learning).

En esta etapa se integraron dos modelos preentrenados modernos para comparar una arquitectura convolucional frente a un Transformer visual bajo el mismo protocolo experimental.

Modelo 1 - EfficientNetB0 (ImageNet): Se utilizó el backbone EfficientNetB0 con pesos de ImageNet completamente congelados. Sobre él se añadió una cabeza de clasificación compuesta por GlobalAveragePooling2D, una capa densa de 256 neuronas con BatchNormalization y Dropout de 0.40, y una capa de salida Softmax.

Modelo 2 - ViT-B16 (Vision Transformer, TF-Hub): Se utilizó el modelo ViT-B16 disponible en TensorFlow Hub como extractor de embeddings en modo no entrenable. Los embeddings extraídos de cada imagen se utilizaron para entrenar una cabeza densa de clasificación con capas de 512 y 256 neuronas, BatchNormalization y Dropout. La elección de TF-Hub responde a criterios de compatibilidad con el entorno Kaggle bajo Keras 3.

Análisis opcionales implementados: comparación cuantitativa entre ambos modelos preentrenados y evaluación de costo computacional (parámetros entrenables y no entrenables) versus ganancia en Macro F1 respecto a la Etapa 2.

4.4 Etapa 4: Componente Generativo

El componente generativo se diseñó para abordar el desbalance de clases del dataset, generando imágenes sintéticas de las clases minoritarias para aumentar su representación durante el entrenamiento. Se implementaron y compararon dos enfoques.

GAN (Generative Adversarial Network): Esquema adversarial compuesto por un generador (dense $\rightarrow$ reshape $\rightarrow$ 3 capas Conv2DTranspose $\rightarrow$ Conv2D con sigmoid) y un discriminador (3 capas Conv2D $\rightarrow$ Flatten $\rightarrow$ capas densas $\rightarrow$ sigmoid). Entrenado con pérdida binary_crossentropy en ambas ramas y optimizador Adam.

VAE (Variational Autoencoder): Encoder convolucional que comprime cada imagen en un vector latente de 32 dimensiones (z_mean y z_log_var), con una capa de muestreo mediante el truco de reparametrización. El decoder reconstruye la imagen desde el espacio latente. La función de pérdida combina pérdida de reconstrucción (MSE) y divergencia KL para regularizar el espacio latente.

Integración: Se identificaron las clases con menor representación relativa (por debajo de la mediana del dataset) y se generaron imágenes sintéticas con cada modelo para balancear esas clases. Se entrenó un clasificador de referencia en tres escenarios comparables: datos originales, datos + imágenes VAE y datos + imágenes GAN. Se compararon los resultados con Macro F1 como métrica principal. Adicionalmente, se exploró el espacio latente del VAE mediante PCA 2D para visualizar la separabilidad entre clases.

5.Experimentos realizados y comparación entre modelos

5.1 Configuración experimental

Para la fase experimental del proyecto se trabajó con el dataset completo Skin Diseases Cancer Comprehensive Dataset, descartando únicamente las clases con menos de 5 muestras. El conjunto de datos se dividió de forma estratificada en 70% entrenamiento, 15% validación y 15% prueba, manteniendo la proporción de clases en cada subconjunto.

Las imágenes se procesaron a distintas resoluciones según la etapa: 192×192 píxeles en Etapa 1, 224×224 píxeles en Etapas 2, 3 y 5, y 128×128 píxeles en Etapa 4. En todas las etapas se trabajó directamente con píxeles RGB normalizados, sin variables tabulares como entrada

al modelo. El desbalance de clases se manejó mediante class weights suavizados con raíz cuadrada, aplicados durante el entrenamiento de todas las etapas.

Los hiperparámetros principales fueron: batch size de 32, hasta 35 épocas con EarlyStopping (patience de 3 a 8 según la etapa) y ReduceLROnPlateau para ajuste automático del learning rate. El optimizador utilizado fue AdamW con clipnorm=1.0 y weight decay variable por etapa. Se utilizó mixed precision (float16) en las etapas 1 y 2 para optimizar el uso de memoria en GPU.

Las métricas de evaluación fueron Accuracy, Precision macro, Recall macro y Macro F1-score, reportadas sobre el conjunto de prueba al finalizar cada etapa.

MLP Base (Etapa 1)

Como línea base se implementó un Perceptrón Multicapa que procesa directamente los píxeles de las imágenes. Las imágenes de entrada se redimensionaron a 192×192 píxeles. Dentro del modelo se aplica reescalado al rango [0,1], augmentación moderada (flip horizontal, rotación, zoom y contraste aleatorio) y un AveragePooling con pool_size=6 para reducir la dimensionalidad antes del bloque denso. La arquitectura consta de tres capas densas de 768, 384 y 192 neuronas con activación ReLU, cada una seguida de BatchNormalization y Dropout (0.45, 0.35 y 0.25 respectivamente). La capa de salida utiliza activación Softmax en float32.

CNN Profunda desde Cero (Etapa 2)

Se implementó una Red Neuronal Convolutiva profunda entrenada sin pesos preentrenados, para aislar el efecto de la arquitectura convolutiva frente al baseline. La arquitectura consta de cuatro bloques convolucionales progresivos con 32, 64, 128 y 256 filtros. Cada bloque aplica dos capas Conv2D (kernel 3×3), BatchNormalization, activación ReLU, MaxPooling2D y Dropout creciente. Una capa Conv2D adicional de 320 filtros precede al GlobalAveragePooling2D. La cabeza de clasificación incluye una capa densa de 256 neuronas con BatchNormalization y Dropout de 0.45. Como análisis opcional se implementaron mapas de activación Grad-CAM sobre la última capa convolutiva.

Modelos Preentrenados (Etapa 3)

Se compararon dos modelos preentrenados bajo el mismo protocolo experimental. EfficientNetB0 se utilizó con pesos de ImageNet completamente congelados, añadiendo una cabeza de clasificación compuesta por GlobalAveragePooling2D, una capa densa de 256 neuronas con BatchNormalization y Dropout de 0.40. ViT-B16 se utilizó como extractor de embeddings desde TensorFlow Hub en modo no entrenable, entrenando sobre sus embeddings una cabeza densa con capas de 512 y 256 neuronas. Se realizó además un análisis de costo computacional comparando parámetros entrenables versus ganancia en Macro F1 respecto a la Etapa 2.

Componente Generativo (Etapa 4)

Se implementaron dos modelos generativos para generar imágenes sintéticas de las clases minoritarias y evaluar su impacto en el clasificador. La GAN consta de un generador con capas Conv2DTranspose y un discriminador con capas Conv2D, entrenados en esquema adversarial con pérdida `binary_crossentropy`. El VAE utiliza un encoder convolucional que comprime cada imagen en un vector latente de 32 dimensiones, con una capa de muestreo por reparametrización, y un decoder que reconstruye la imagen desde ese espacio latente. La pérdida combina reconstrucción MSE y divergencia KL. Se entrenó un clasificador de referencia en tres escenarios: datos originales, datos aumentados con VAE y datos aumentados con GAN, comparando los resultados con Macro F1.

Fine-Tuning y Despliegue (Etapa 5)

Se aplicó fine-tuning en dos fases sobre EfficientNetB0. En la primera fase, con la base completamente congelada, se entrenó únicamente la cabeza de clasificación durante 5 épocas con learning rate de $3e-4$. En la segunda fase se descongelaron las últimas 30 capas de la base y se recompiló con learning rate reducido de $5e-5$, realizando ajustes finos durante hasta 10 épocas adicionales con EarlyStopping y ReduceLROnPlateau. El sistema fue publicado como notebook ejecutable en Kaggle, plataforma pública con soporte GPU donde cualquier usuario puede reproducir los experimentos e interactuar con el modelo.

5.2 Resultados reales obtenidos

Los resultados registrados en el notebook fueron los siguientes:

Modelo	Accuracy	Precision Macro	Recall Macro	Macro F1
Eta p a 1 — MLP base (imágenes)	0.2618	0.2125	0.2565	0.2152
Eta p a 2 — CNN profunda desde cero	0.3957	0.4019	0.3973	0.3828
Eta p a 3 — EfficientNetB0 (preentrenado)	0.5366*	—	—	—
Eta p a 3 — ViT-B16 (preentrenado)	0.5864	0.5816	0.5947	0.5836
Eta p a 4 — Clasificador ViT baseline	—	—	—	0.6039
Eta p a 5 — Fine-tuning parcial ViT	0.7040	0.7077	0.7018	0.7025

*Accuracy de validación final de EfficientNetB0 al completar las 12 épocas. El mejor modelo de la etapa fue ViT-B16.

Adicionalmente, en la Etapa 5 se evaluaron tres estrategias opcionales de optimización sobre el modelo de fine-tuning:

Método	Accuracy	Macro F1	Delta F1 vs baseline
Fine-tuning parcial (baseline)	0.7040	0.7025	—
FP16 inference (autocast)	0.7040	0.7025	0.0000
Pruning 20% capas lineales	0.6927	0.6902	-0.0124
Cuantización INT8 dinámica	0.6646	0.6639	-0.0386

5.3 Análisis comparativo de resultados

Los resultados obtenidos evidencian una mejora progresiva y consistente a lo largo de las cinco etapas del proyecto, confirmando la validez de la metodología incremental adoptada.

El MLP base de la Etapa 1 alcanzó una accuracy de 26.18% y un Macro F1 de 0.2152. Aunque modesto, este resultado era esperado dado que un perceptrón multicapa no está diseñado para capturar patrones espaciales en imágenes. Su valor principal fue establecer una referencia cuantitativa clara para las etapas siguientes.

La CNN profunda de la Etapa 2, entrenada desde cero, mejoró notablemente sobre el baseline al alcanzar una accuracy de 39.57% y un Macro F1 de 0.3828, representando una mejora de aproximadamente +0.17 puntos en F1. Este resultado confirma que la arquitectura convolucional, al extraer características espaciales locales como bordes, texturas y patrones visuales, es significativamente más adecuada para la clasificación de imágenes dermatológicas que un MLP puro.

La incorporación de modelos preentrenados en la Etapa 3 representó el salto de desempeño más significativo del proyecto. EfficientNetB0 con pesos de ImageNet mejoró considerablemente respecto a la CNN desde cero, pero fue superado por ViT-B16, que alcanzó una accuracy de 58.64% y un Macro F1 de 0.5836, una mejora de aproximadamente +0.20 puntos sobre la Etapa 2. Este resultado respalda la hipótesis de que el mecanismo de self-attention del Transformer visual, al capturar relaciones globales entre regiones de la imagen, ofrece ventajas sobre las arquitecturas puramente convolucionales en este dominio clínico.

En la Etapa 4, el componente generativo con ViT como backbone de clasificación consolidó un Macro F1 de 0.6039, superando por primera vez la meta de referencia de 0.60 establecida en la rúbrica. Los modelos generativos VAE y GAN permitieron sintetizar imágenes de las 12 clases minoritarias del dataset, contribuyendo al equilibrio del entrenamiento.

Finalmente, el fine-tuning parcial de la Etapa 5 produjo el mejor desempeño global del sistema, con una accuracy de 70.40% y un Macro F1 de 0.7025, superando en más de +0.10 puntos el resultado de la Etapa 3. Descongelar las últimas capas del ViT con un learning rate

reducido permitió adaptar las representaciones aprendidas al dominio específico de enfermedades de piel sin perder el conocimiento previo adquirido en el preentrenamiento. Entre las estrategias opcionales de optimización evaluadas, FP16 inference mantuvo el mismo desempeño que el baseline sin pérdida alguna de calidad, mientras que pruning e INT8 introdujeron reducciones menores aceptables en escenarios donde el tiempo de inferencia o el tamaño del modelo sean prioritarios.

6. Bibliografía

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Yann LeCun, Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.
- Alex Krizhevsky, Sutskever, I., & Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. Advances in Neural Information Processing Systems.
- Mingxing Tan & Quoc V. Le (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. Proceedings of the 36th International Conference on Machine Learning.